⇒ The Final Linear & Softmax Layer.

↳ The decoder stack outputs a vector of floats. How do we turn that into a word? That's the job of the final Linear layer which is followed by a Softmax Layer.

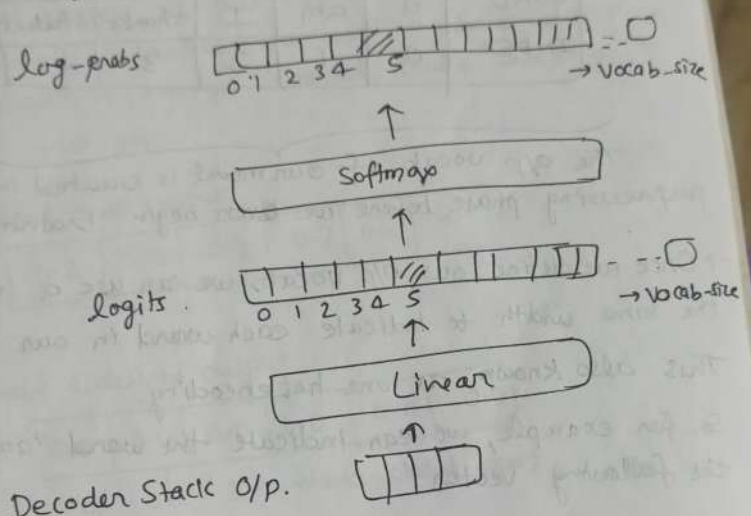
↳ The Linear layer is a simple fully connected neural n/w that projects the vectors produced by the stack of decoders, into a much, much larger vector called a logits vector

↳ Let's assume that our model knows 10,000 unique English words (our model's "output vocabulary") that it's learned from its training dataset. This would make the logits vector 10,000 cells wide - each cell corresponding to the score of a unique word. That is how we interpret the output of the model followed by the Linear layer.

The softmax layer then turns those scores into probs (all positive, all add up to 1.0). The cell with the highest prob. is chosen & the word associated with it is produced as the o/p for this time step.

Which word in our vocab is associated with this index? am

Get the index of the cell with the highest value (argmax) 5



⇒ Recap of Traing.

Now that we've covered the entire forward-pass process through a trained Transformer, it would be useful to glance at the intuition of training the model.

During training, an untrained model would go through the exact same forward pass. But since we are training it on a labeled training dataset, we can compare its o/p with the actual correct o/p.

→ To visualize this, let's assume our o/p vocabulary only contains 6 words ("a", "am", "I", "thanks", "student", and "<eos>" ('end of sentence')).

o/p vocab

WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

The o/p vocab of our model is created in the preprocessing phase before we even begin training.

→ Once we define our o/p vocab, we can use a vector of the same width to indicate each word in our vocab. This also known as one-hot encoding.

So, for example, we can indicate the word "am" using the following vector:

a	am	I	thanks	student	<eos>
0.0	1.0	0.0	0.0	0.0	0.0

Following this recap, let's discuss the model's loss function the metric we are optimizing during the training phase to lead up to a trained & hopefully amazingly accurate model.

⇒ The Loss

Say we are in the train example — What this ha a prob. dist this mode happen J

Untrained

Correct

Since initiali a prob We car all th make

→ The Loss Function

Say we are training our model. Say it's our 1st step in the training phase & we're training it on a simple example – translating "merci" to "thanks".

What this happens, is that we want the output to be a prob. distn indicating the word "thanks". But since this model is not yet trained, that's unlikely to happen just yet.

Untrained Model Output

0.2	0.2	0.1	0.2	0.2	0.1

Correct & desired output

0.0	0.0	0.0	1.0	0.0	0.0

a am I thanks student <eos>

Since the model's parameters (weights) are all initialized randomly, the (untrained) model produces a prob. distn with arbitrary values for each cell/word. We can compare it with the actual O/P, then tweak all the model's weights using backpropagation to make the O/P closer to the desired O/P.

↳ How do you compare two prob. dist^{n}s ? We simply subtract one from the other. For more details, look at cross-entropy & kullback - Leibler divergence.

↳ But note that this is an oversimplified example. More realistically, we'll use a sentence longer than one word. For example - input: "Je suis étudiant" & expected o/p: "i am a student". What this really means, is that we want our model to successively output prob. dist^n where:

- Each prob. dist^n is represented by a vector of width vocab-size (6 in our toy example, but more realistically a number like 30,000 or 50,000)
- The 1st prob. dist^n has the highest prob. at the cell associated with the word "i"
- The 2nd prob dist^n has the highest prob. at the cell associated with the word "am".
- And so on, until the 5th o/p dist^n indicates '<end of sentence>' symbol which also has a cell associated with it from the 10,000 element vocabulary.

Target m

Output voca

Position #

Position #

Position #

Position

Position

After trai
enough da
dist^{n}s wo

↳ Trained m

O/p Voca

position

Position

Position

Position

Posiff

Target Model Outputs.

Output Vocab:	a	am	I	thanks	student	<eos>
Position #1	0.0	0.0	1.0	0.0	0.0	0.0
Position #2	0.0	1.0	0.0	0.0	0.0	0.0
Position #3	1.0	0.0	0.0	0.0	0.0	0.0
Position #4	0.0	0.0	0.0	0.0	1.0	0.0
Position #5	0.0	0.0	0.0	0.0	0.0	1.0

After training the model for enough time on a large enough dataset, we would hope the produced prob. dist^{n}s would look like this:

↳ Trained Model O/Ps

O/P Vocab	a	am	I	thanks	student	<eos>
position #1	0.01	0.02	0.93	0.01	0.03	0.01
Position #2	0.01	0.8	0.1	0.05	0.01	0.03
Position #3	0.99	0.001	0.001	0.001	0.002	0.001
Position #4	0.001	0.002	0.001	0.02	0.94	0.01
Position #5	0.01	0.01	0.001	0.001	0.01	0.98
	a	am	I	thanks	student	<eos>

Now, because the model produces the outputs one at a time, we can assume that the model is selecting the word with the highest prob. from that prob. distn & throwing away the rest. That's one way to do it (called greedy decoding). Another way to do it would be to hold onto, say, the top 2 words (say, 'I' and 'a' for example), then in the next step, run the model twice: once assuming the 1st output position was the word 'I' & another time assuming the 1st o/p position was the word 'a' & whichever version produced less error considering both positions #1 & #2 is kept. We repeat this for positions #2 & #3 ...etc. This method is called "beam search", where in our example, beam size was 2 (meaning that at all times, 2 partial hypotheses (unfinished translations) are kept in memory), and top-beams is also two (meaning we'll return 2 translations). These are both hyperparameters that you can experiment with.

(*) How GPT 3 Works – Visualizations & Animations

↳ GPT3 is a massive language model.

→ A trained language model generates text.

→ We can optionally pass it some text as input, which influences its output. The output is generated from what the model "learned" during its training period where it scanned vast amounts of text.

Ex

Input Prompt: Recite the first law of robotics

↓

GPT-3

↓

Output: A robot may not injure a human being or, through inaction, allow a human being to come to harm.

↳ Training is the process of exposing the model to lots of text. That process has been completed. All the experiments you see now are from that one trained model. It was estimated to cost 355 GPU years & cost $4.6m.

Untrained GPT-3

⇓

Unsupervised Pre-training

Untrained GPT-3

Expensive training on massive datasets.

Dataset: 300 billion tokens of text.

Objective: Predict next word.

Ex. a robot must ?

⇓

GPT-3.

The dataset of 300 billion tokens of text is used to generate training examples for the model. For ex., these are 3 training examples generated from one sentence at the top.

↳ You can see how you can slide a window across all the text & makes lots of examples.

Text: Second Law of Robotics: A robot must obey the orders given it by human beings.

⇓ Generated training examples

Example #	Input (features)	Correct output (label)
1	Second law of robotics :	a
2	Second law of robotics : a	robot
3	Second law of robotics : a robot	must

...

The model is pre
features & ask it
The model's predi
error in its
time it makes
→ Repeat millions
Unsupervis
Input (feat
update
trainin
&
model
gets
upd

↳ Now, let's l
detail.

GPT3 actuall
(let's assume